AGENDA 8

UTILIZANDO MICROSOFT MAUI PARA DESENVOLVIMENTO MOBILE

GEEaD - Grupo de Estudos de Educação a Distância

Centro de Educação Tecnológica Paula Souza

GOVERNO DO ESTADO DE SÃO PAULO

EIXO TECNOLÓGICO DE INFORMAÇÃO E COMUNICAÇÃO

CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

PROGRAMAÇÃO MOBILE II

Expediente

Autores:

TIAGO ANTONIO DA SILVA

Revisão Técnica:

KELLY CRISTIANE DE OLIVEIRA DAL POZZO

GUILHERME HENRIQUE GIROLLI

São Paulo – SP, 2024

Com o avanço da tecnologia embarcada nos smartphones, os dispositivos móveis se tornaram verdadeiros centros de coleta de dados sobre o ambiente ao seu redor. Parte fundamental dessa evolução está nos sensores integrados ao hardware dos dispositivos.

Sensores em Dispositivos Móveis

Sensores são componentes físicos capazes de detectar mudanças no ambiente ao redor do dispositivo e fornecer esses dados ao sistema operacional e aos aplicativos. Esses dados podem incluir movimento, orientação, posição geográfica, proximidade, luminosidade, entre outros.

Os sensores permitem que os aplicativos ofereçam experiências mais interativas, adaptáveis e inteligentes. Por exemplo:

- Um app de saúde pode contar os passos do usuário usando o acelerômetro.
- Um app de mapas pode determinar a direção do usuário com o magnetômetro.
- Um app de câmera pode ajustar o foco com base na proximidade.

Sensores comuns em dispositivos móveis

Sensor	Finalidade
Acelerômetro	Detecta movimento e inclinação do dispositivo.
Giroscópio	Mede a rotação e orientação.
Magnetômetro	Detecta campos magnéticos; usado como bússola.
GPS	Fornece localização geográfica (latitude, longitude).
Sensor de luz	Mede a intensidade da luz ambiente.
Sensor de proximidade	Detecta a proximidade de objetos sem contato físico.

Sensores e o .NET MAUI

O .NET MAUI fornece uma API unificada e simplificada para acessar sensores em diferentes plataformas (Android, iOS, Windows e macOS). Com ele, é possível acessar sensores nativos de maneira simples e com código compartilhado entre plataformas.

A localização geográfica (GPS) é frequentemente tratada como um sensor, mas no .NET MAUI ela é acessada por meio de uma API distinta (Geolocation), localizada no namespace Microsoft.Maui.Devices.Geolocation. Apesar disso, seu uso está diretamente relacionado ao contexto de sensores e será abordado nesta agenda como parte do conjunto de funcionalidades voltadas à percepção do ambiente pelo dispositivo.

Obtendo a localização do Dispositivo

No .NET MAUI, a API de Geolocalização permite que os aplicativos obtenham a localização do dispositivo, seja por GPS, Wi-Fi ou redes móveis. Isso é essencial para funcionalidades como rastreamento, mapas, serviços baseados em localização e recomendações geográficas. Os principais recursos são:

- Obter coordenadas (latitude/longitude);
- Converter coordenadas em endereços (Geocoding);
- Converter endereços em coordenadas;
- Monitorar a localização em tempo real.

Entretanto, é importante ficar atento ao funcionamento do App, pois a Geolocalização precisa de permissões do usuário para funcionar e assim estudaremos no item a seguir como solicitar essas permissões ao usuário sem interromper o bom funcionamento do App.

GeolocationRequest

Ao trabalhar com localização geográfica no .NET MAUI, utiliza-se a classe **Geolocation** para obter a posição atual do dispositivo (latitude, longitude e outros dados). No entanto, para definir como essa localização deve ser obtida, utiliza-se a classe GeolocationRequest.

A classe GeolocationRequest permite configurar o modo e a precisão com que a localização será obtida. Ela é usada como parâmetro no método GetLocationAsync() da classe Geolocation.

Veja um exemplo:

```
var request = new GeolocationRequest(
    GeolocationAccuracy.High,
    TimeSpan.FromSeconds(10));

Location location = await Geolocation.GetLocationAsync(request);

if (location != null)
{
    Console.WriteLine($"Latitude: {location.Latitude}, Longitude: {location.Longitude}");
}
```

Onde:

Propriedade	Descrição
Accuracy	Define o nível de precisão desejado (GeolocationAccuracy.High)
Timeout	Define o tempo máximo de espera pela localização (como um TimeSpan)

A classe Geolocation faz parte do namespace Microsoft.Maui.Devices.Sensors e é usada para obter a localização do dispositivo. O método **GetLocationAsync()** obtém a localização atual do dispositivo com base nas configurações padrão. O método retorna um objeto Location contendo latitude, longitude, altitude, velocidade e outros dados. Pode lançar exceções, como PermissionException ou FeatureNotEnabledException.

GetLocationAsync(GeolocationRequest request) que obtém a localização atual do dispositivo com um pedido personalizado, permitindo definir a precisão e o tempo máximo de espera. Veja um exemplo:

```
var request = new GeolocationRequest(GeolocationAccuracy.High,TimeSpan.FromSeconds(10));
Location location = await Geolocation.GetLocationAsync(request);
```

O Parâmetro GeolocationRequest permite ajustar a precisão (GeolocationAccuracy.Default, Low, Medium, High, Best) e definir um timeout (tempo máximo de espera) em segundos.

Tratando problemas na obtenção da localização

Ao utilizar a classe GeolocationRequest é importante tratar exceções para garantir uma melhor experiência e evitar falhas inesperadas. A API pode lançar várias exceções dependendo do cenário.

```
try
{
    var request = new GeolocationRequest(GeolocationAccuracy.High,TimeSpan.FromSeconds(10));
    Location location = await Geolocation.GetLocationAsync(request);

    if (location != null)
    {
        Console.WriteLine($"Latitude: {location.Latitude}, Longitude:{location.Longitude}");
    }
}
catch (FeatureNotSupportedException fnsEx)
{
    // O dispositivo não suporta geolocalização
    Console.WriteLine("Erro: Geolocalização não é suportada neste dispositivo.");
}
catch (FeatureNotEnabledException fneEx)
{
    //GPS/Localização está desativado
    Console.WriteLine("Erro: A localização está desativada. Ative o GPS e tente
novamente.");
}
```

```
catch (PermissionException pEx)
{
    //Permissão negada pelo usuário
    Console.WriteLine("Erro: Permissão de localização negada. Conceda a permissão nas
configurações do dispositivo.");
}
catch (TimeoutException tEx)
{
    //Tempo limite excedido ao tentar obter localização
    Console.WriteLine("Erro: Tempo limite atingido ao tentar obter a localização. Tente
novamente.");
}
catch (Exception ex)
{
    //Outro erro inesperado
    Console.WriteLine($"Erro desconhecido: {ex.Message}");
}
```

Vamos entender as exceções que podem ser tratadas:

- **FeatureNotSupportedException:** O dispositivo não suporta GPS/Geolocalização. Verifique se o dispositivo tem suporte antes de solicitar a localização.
- **FeatureNotEnabledException**: O GPS está desativado. Peça ao usuário para ativar o GPS.
- **PermissionException**: O usuário negou a permissão. Solicite novamente ou informe como ativar nas configurações.
- **TimeoutException:** O GPS não conseguiu obter a localização a tempo. Aumente o tempo de espera ou use outra precisão (GeolocationAccuracy).
- **Exception:** Qualquer outro erro inesperado. Exibir uma mensagem amigável ao usuário e registrar o erro.

Essas são algumas dicas de como podemos tratar as exceções quando solicitamos a localização de um dispositivo, mas também considere as boas práticas:

- Sempre solicite permissões antes de chamar GetLocationAsync().
- Use try-catch para evitar falhas inesperadas no App.
- Defina um tempo limite adequado para evitar que o app trave.
- Considere mostrar um alerta ao usuário se a permissão for negada ou o GPS estiver desligado.

Detalhes de localização com Placemark

A API de Geocoding permite converter coordenadas (latitude e longitude) em endereços legíveis e vice-versa. Isso é feito através dos Placemarks e Locations. O Placemark retorna informações detalhadas sobre o local:

- **Thoroughfare:** Nome da rua;
- **SubThoroughfare:** Número da casa;

- **Locality:** Cidade;
- **AdminArea:** Estado/Província;
- **CountryName:** País;
- **PostalCode:** CEP.

A classe Placemark pode ser útil:

- Exibir a localização do usuário em um mapa;
- Mostrar o endereço de um ponto GPS;
- Validar um endereço digitado pelo usuário;
- Criar um sistema de entrega baseado em geolocalização.

No projeto apresentado duranta a videoaula, a classe Placemark foi utilizada dentro do método privado GetCidade(double lat, double lon), com o objetivo de converter as coordenadas geográficas (latitude e longitude) em um nome de localidade (cidade). Veja:

```
private async void GetCidade(double lat, double lon)
{
    try
    {
        IEnumerable<Placemark> places = await Geocoding.Default.GetPlacemarksAsync(lat, lon);

        Placemark? place = places.FirstOrDefault();

        if (place != null)
        {
            txt_cidade.Text = place.Locality;
        }
    }
    catch(Exception ex)
    {
        await DisplayAlert("Erro: Obtenção do nome da Cidade", ex.Message, "OK");
    }
}
```

Algumas Observações Importantes:

- *A disponibilidade das propriedades pode variar de acordo com o sistema operacional e a fonte de dados.*
- *Pode ser necessário testar a aplicação em um dispositivo real para obter resultados completos (nem todos os emuladores retornam dados válidos).*
- *É importante sempre verificar se o Placemark retornado não é null.*

WebView

O WebView é um componente visual utilizado para exibir conteúdo da web diretamente dentro de um aplicativo. Com ele, é possível carregar páginas HTML, sites, ou até mesmo conteúdo local (como arquivos HTML armazenados no próprio aplicativo), tudo isso sem a necessidade de abrir um navegador externo. No .NET MAUI, o WebView está disponível como um controle da interface gráfica e pode ser usado em qualquer página do App.

O WebView serve para integrar conteúdo da web diretamente dentro do aplicativo. Com ele, é possível mostrar o conteúdo de um site sem sair do App, exibir páginas de ajuda ou documentação hospedadas online, rodar aplicações web desenvolvidas com HTML, CSS e JavaScript no ambiente de um app nativo e até incorporar widgets de terceiros, como mapas interativos, vídeos e formulários online.

Principais Características:

- Suporte a HTML, CSS e JavaScript diretamente na aplicação.
- Capacidade de carregar URLs remotas e arquivos locais.
- Comunicação bidirecional entre JavaScript e C#.
- Suporte a eventos, como navegação iniciada, concluída e erros.

No projeto MauiAppTempoAgora, o site Windy.com (https://www.windy.com/) foi aplicado dentro do componente WebView da MainPage, sendo usado para exibir um mapa meteorológico interativo com base na localização da cidade consultada.

O uso ocorre dentro do método Button_Clicked_Previsao, no arquivo MainPage.xaml.cs:

```
string mapa = $"https://embed.windy.com/embed.html?" +
              $"type=map&location=coordinates&metricRain=mm&metricTemp=°C" +
              $"&metricWind=km/h&zoom=5&overlay=wind&product=ecmwf&level=surface" +
              $"&lat={t.lat.ToString().Replace(",", ".")}&lon={t.lon.ToString().Replace(",",
".")}";

wv_mapa.Source = mapa;
```

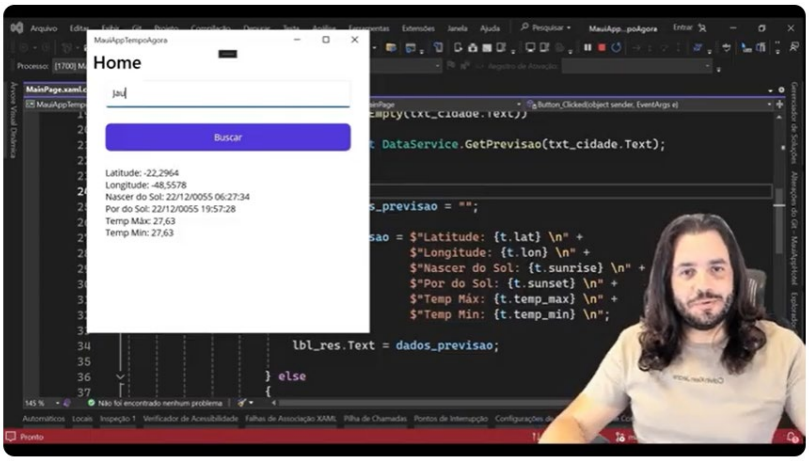
O método **Replace(",", ".")** está tentando forçar o formato decimal para o padrão de países que usam ponto como separador decimal (como o esperado pela URL do Windy). No entanto, isso depende da cultura local da máquina e pode causar problemas se o app estiver sendo executado em dispositivos com configurações de idioma diferentes (por exemplo, pt-BR usa vírgula).

A abordagem mais robusta é usar uma cultura invariável, garantindo que o número será formatado com ponto decimal independentemente da localização.

```
string mapa = $"https://embed.windy.com/embed.html?" +
              $"type=map&location=coordinates&metricRain=mm&metricTemp=°C" +
              $"&metricWind=km/h&zoom=5&overlay=wind&product=ecmwf&level=surface" +
              $"&lat={t.lat.ToString("F6",CultureInfo.InvariantCulture)}&lon={t.lon.ToString
("F6", CultureInfo.InvariantCulture)}";

wv_mapa.Source = mapa;
```

Assista o vídeo apresentando o passo a passo para criação do aplicativo:



Disponível em: https://www.youtube.com/watch?v=smmaNhkCzfk

CÓDIGO-FONTE DA AGENDA:

https://github.com/tiagotas/MauiAppTempoAgora/tree/4a79197e86aeb2bcd8c06ce2cfd2a533274af1fa